

Job 01 . Installation et configuration des VMs Debian + K3S

Pré-requis

Distribution	Debian 12 (sans GUI)
Hyperviseur	VMware Workstation
Distribution K8s	K3S
Réseau cluster	Host-Only — 172.16.227.0/24
Réseau internet	NAT — 172.16.197.0/24

Architecture réseau

VM	Hostname	IP Host-Only (ens34)	IP NAT (ens32)	Rôle K3S
kubes-01	kubes-01.local	172.16.227.10	172.16.197.131	Master
kubes-02	kubes-02.local	172.16.227.11	172.16.197.132	Worker
kubes-03	kubes-03.local	172.16.227.12	172.16.197.133	Worker

1. Configuration réseau VMware

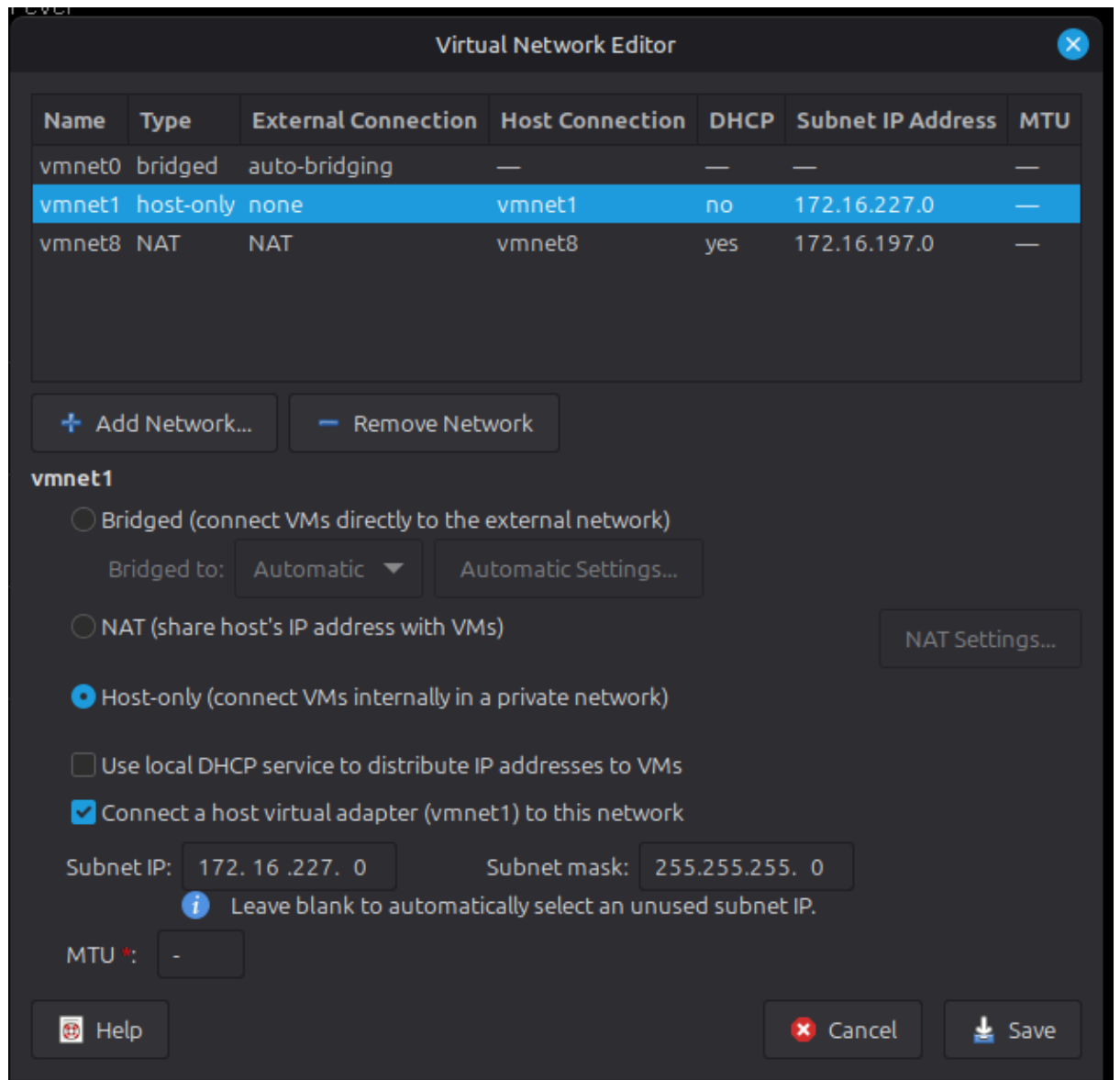
Réseau Host-Only (VMnet1)

Dans VMware Workstation : [Édition](#) → [Virtual Network Editor](#)

- VMnet1 → Host-only, sous-réseau [172.16.227.0/255.255.255.0](#), DHCP désactivé
- VMnet8 → NAT (accès internet)

Chaque VM dispose de deux cartes réseau :

- Adaptateur 1 → NAT (VMnet8)
- Adaptateur 2 → Host-only (VMnet1)



2. Configuration des VMs (répétée sur les 3)

2.1 Hostname

hostnamectl set-hostname kubens-01.local # faire pareil sur chaque VM

```

root@kubes-01:~/deployments# hostnamectl
Static hostname: kubes-01.local
    Icon name: computer-vm
    Chassis: vm 🖥️
    Machine ID: 63f4c95e4efaf65dbb0b28116a26c9ad
    Boot ID: 40da8ba2747345df93384154a478bac5
    Product UUID: ce9f4d56-abcc-9432-edd4-28a1465c489e
    AF_VSOCK CID: 1180453022
Virtualization: vmware
Operating System: Debian GNU/Linux 13 (trixie)
    Kernel: Linux 6.12.90+deb13.1-amd64
    Architecture: x86-64
Hardware Vendor: VMware, Inc.
Hardware Model: VMware Virtual Platform
Hardware Serial: VMware-56 4d 9f ce cc ab 32 94-ed d4 28 a1 46 5c 48 9e
Firmware Version: 6.00
    Firmware Date: Mon 2025-03-24
    Firmware Age: 1y 2month 2w 2d

```

2.2 Fichier /etc/hosts

```

cat > /etc/hosts << EOF
127.0.0.1    localhost
127.0.1.1    kubes-01.local kubes-01

::1         localhost ip6-localhost ip6-loopback
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters

172.16.227.10 kubes-01.local kubes-01
172.16.227.11 kubes-02.local kubes-02
172.16.227.12 kubes-03.local kubes-03
EOF

```

2.3 Interface réseau Host-Only dans /etc/network/interfaces

```

cat >> /etc/network/interfaces << EOF
auto ens34
iface ens34 inet static
    address 172.16.227.10 # .11 sur kubes-02, .12 sur kubes-03
    netmask 255.255.255.0
EOF

```

Puis afficher : *ifup ens34*

```

root@kubes-01:~/deployments# ip a show ens34
2: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:5c:48:a8 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    altname enx000c295c48a8
    inet 172.16.227.10/24 brd 172.16.227.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe5c:48a8/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

```

```

root@kubes-02:~# ip a show ens34
2: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:d9:c5:f1 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    altname enx000c29d9c5f1
    inet 172.16.227.11/24 brd 172.16.227.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fed9:c5f1/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

```

```

root@kubes-03:~# ip a show ens34
2: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:e7:ee:fc brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    altname enx000c29e7eefc
    inet 172.16.227.12/24 brd 172.16.227.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fee7:ee7c/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

```

2.4 Prérequis système

Mise à jour et installation des outils:

```

apt update && apt upgrade -y
apt install -y curl wget vim

```

Désactivation du swap (obligatoire pour K3S/Kubernetes)

```

swapoff -a
sed -i ' / swap / s/^/#/' /etc/fstab

```

Vérification : `free -h`

```

total    utilisé    libre    partagé    tamp/cache    disponible
Mem:      1,9Gi    412Mi    667Mi    652Ki    1,0Gi    1,5Gi
Échange:    0B      0B      0B

```

3. Test de connectivité inter-VMs

Depuis kubes-01 :

```

ping -c 3 172.16.227.11 # vers kubes-02
ping -c 3 172.16.227.12 # vers kubes-03

```

```
root@kubes-01:~/deployments# ping -c 3 172.16.227.11
PING 172.16.227.11 (172.16.227.11) 56(84) bytes of data.
64 bytes from 172.16.227.11: icmp_seq=1 ttl=64 time=2.11 ms
64 bytes from 172.16.227.11: icmp_seq=2 ttl=64 time=1.90 ms
64 bytes from 172.16.227.11: icmp_seq=3 ttl=64 time=1.64 ms

--- 172.16.227.11 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.643/1.884/2.110/0.191 ms
```

```
root@kubes-01:~/deployments# ping -c 3 172.16.227.12
PING 172.16.227.12 (172.16.227.12) 56(84) bytes of data.
64 bytes from 172.16.227.12: icmp_seq=1 ttl=64 time=1.95 ms
64 bytes from 172.16.227.12: icmp_seq=2 ttl=64 time=1.46 ms
64 bytes from 172.16.227.12: icmp_seq=3 ttl=64 time=1.45 ms

--- 172.16.227.12 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 1.453/1.619/1.946/0.231 ms
```

4 .Installation K3S (mode standalone)

K3S est installé en mode standalone sur chaque VM.

- Sur kubes-01

```
curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="--node-ip=172.16.227.10 \  
--advertise-address=172.16.227.10 \  
--flannel-iface=ens34" sh -
```

- Sur kubes-02

```
curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="--node-ip=172.16.227.11 \  
--advertise-address=172.16.227.11 \  
--flannel-iface=ens34" sh -
```

- Sur kubes-03

```
curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="--node-ip=172.16.227.12 \  
--advertise-address=172.16.227.12 \  
--flannel-iface=ens34" sh -
```

L'option `--flannel-iface=ens34` force K3S à utiliser l'interface Host-Only plutôt que l'interface NAT pour la communication inter-pods.

5.Vérification K3S

Pour vérifier que le service tourne : *systemctl status k3s*

```
laplateforme@kubes-02:~$ systemctl status k3s
[INFO] systemd: Starting k3s
root@kubes-02:~# systemctl status k3s
● k3s.service - Lightweight Kubernetes
   Loaded: loaded (/etc/systemd/system/k3s.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-08 15:55:02 CEST; 1min 29s ago
 Invocation: 4857d9fa26034c3e8e92936936f623f1
    Docs: https://k3s.io
   Process: 15161 ExecStartPre=/sbin/modprobe br_netfilter (code=exited, status=0/SUCCESS)
   Process: 15162 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCESS)
   Main PID: 15165 (k3s-server)
      Tasks: 82
     Memory: 1.2G (peak: 1.2G)
        CPU: 2min 31.377s
    CGroup: /system.slice/k3s.service
            └─15165 "/usr/local/bin/k3s server"
               └─15219 "containerd"
                  └─15740 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b23ec9aed3aa
                     └─15811 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b23ec9aed3aa
                        └─15921 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b23ec9aed3aa
                           └─16015 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b23ec9aed3aa
                              └─16210 /var/lib/rancher/k3s/data/921c0d40b0b3f73f2beae03ec1b23ec9aed3aa

juin 08 15:56:27 kubes-02.local k3s[15165]: I0608 15:56:27.760606 15165 handler.go:
juin 08 15:56:27 kubes-02.local k3s[15165]: I0608 15:56:27.844669 15165 reconciler
juin 08 15:56:27 kubes-02.local k3s[15165]: I0608 15:56:27.844877 15165 reconciler
juin 08 15:56:27 kubes-02.local k3s[15165]: I0608 15:56:27.844920 15165 reconciler
juin 08 15:56:27 kubes-02.local k3s[15165]: I0608 15:56:27.844948 15165 reconciler
juin 08 15:56:28 kubes-02.local k3s[15165]: I0608 15:56:28.624349 15165 scope.go:12
juin 08 15:56:28 kubes-02.local k3s[15165]: I0608 15:56:28.636854 15165 scope.go:12
juin 08 15:56:28 kubes-02.local k3s[15165]: E0608 15:56:28.637839 15165 pod_workers
juin 08 15:56:28 kubes-02.local k3s[15165]: I0608 15:56:28.972898 15165 handler.go:
juin 08 15:56:28 kubes-02.local k3s[15165]: I0608 15:56:28.973154 15165 handler.go:

root@kubes-02:~# kubectl get nodes
NAME                 STATUS   ROLES    AGE   VERSION
kubes-02.local       Ready    control-plane 100s   v1.35.5+k3s1
root@kubes-02:~#
```

Pour vérifier que le node est Ready : *kubectl get nodes*

```
root@kubes-01:~/deployments# kubectl get nodes
NAME                 STATUS   ROLES    AGE   VERSION
kubes-01.local       Ready    control-plane 22h   v1.35.5+k3s1
kubes-02.local       Ready    <none>    3h40m v1.35.5+k3s1
kubes-03.local       Ready    <none>    3h37m v1.35.5+k3s1
```

Job 02.Déploiement des applications conteneurisées

Déploiement de 3 applications conteneurisées sur chaque VM K3S standalone :

- **Nginx** — serveur web (NodePort 30080)
- **Apache (httpd)** — serveur web (NodePort 30081)
- **MariaDB** — base de données (NodePort 30306)

Structure des fichiers

```
~/deployments/
├── nginx.yaml
├── apache.yaml
└── mariadb.yaml
```

mkdir ~/deployments && cd ~/deployments

Fichiers YAML de déploiement

nginx.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      nodePort: 30080
```

apache.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache
spec:
  replicas: 1
  selector:
    matchLabels:
```

```
  app: apache
template:
  metadata:
    labels:
      app: apache
  spec:
    containers:
      - name: apache
        image: httpd:latest
        ports:
          - containerPort: 80
```

```
apiVersion: v1
kind: Service
metadata:
  name: apache-svc
spec:
  type: NodePort
  selector:
    app: apache
  ports:
    - port: 80
      nodePort: 30081
```

mariadb.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mariadb
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
        - name: mariadb
          image: mariadb:latest
          env:
```

```
- name: MYSQL_ROOT_PASSWORD
  value: "rootpassword"
ports:
- containerPort: 3306
```

apiVersion: v1

kind: Service

metadata:

name: mariadb-svc

spec:

type: NodePort

selector:

app: mariadb

ports:

- port: 3306

nodePort: 30306

Déploiement

kubectl apply -f nginx.yaml

kubectl apply -f apache.yaml

kubectl apply -f mariadb.yaml

```
spec:
  type: NodePort
  selector:
    app: mariadb
  ports:
    - port: 3306
      nodePort: 30306
EOF
root@kubes-01:~/deployments# kubectl apply -f nginx.yaml
kubectl apply -f apache.yaml
kubectl apply -f mariadb.yaml
deployment.apps/nginx created
service/nginx-svc created
deployment.apps/apache created
service/apache-svc created
deployment.apps/mariadb created
service/mariadb-svc created
root@kubes-01:~/deployments# kubectl get pods -w
NAME                                READY   STATUS              RESTARTS   AGE
apache-84647dcc49-sk7cp             0/1     ContainerCreating   0           21s
mariadb-c68cf7b8c-2xx25             0/1     ContainerCreating   0           19s
nginx-59f86b59ff-m9t9j             0/1     ContainerCreating   0           23s
apache-84647dcc49-sk7cp             1/1     Running             0           32s
nginx-59f86b59ff-m9t9j             1/1     Running             0           38s
mariadb-c68cf7b8c-2xx25             1/1     Running             0           45s
^C
root@kubes-01:~/deployments# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
apache-84647dcc49-sk7cp             1/1     Running   0           6m42s
mariadb-c68cf7b8c-2xx25             1/1     Running   0           6m40s
nginx-59f86b59ff-m9t9j             1/1     Running   0           6m44s
root@kubes-01:~/deployments# kubectl get services
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
apache-svc  NodePort    10.43.10.123   <none>          80:30081/TCP     6m54s
kubernetes  ClusterIP   10.43.0.1      <none>          443/TCP          34m
mariadb-svc NodePort    10.43.180.210 <none>          3306:30306/TCP   6m51s
nginx-svc   NodePort    10.43.70.222   <none>          80:30080/TCP     6m55s
root@kubes-01:~/deployments#
```

Suivi du démarrage : *kubect! get pods -w*

```
root@kubes-01:~/deployments# kubect! get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
apache-84647dcc49-b65cp             1/1     Running   0           57m
apache-84647dcc49-jd9rk             1/1     Running   0           3h16m
apache-84647dcc49-vzwq6             1/1     Running   0           3h16m
mariadb-c68cf7b8c-87bnx             1/1     Running   0           3h16m
mariadb-c68cf7b8c-c698t             1/1     Running   0           3h16m
mariadb-c68cf7b8c-xvn75             1/1     Running   0           57m
nginx-59f86b59ff-9qdps              1/1     Running   0           3h10m
nginx-59f86b59ff-hlldv              1/1     Running   0           57m
nginx-59f86b59ff-rwm8n              1/1     Running   0           3h10m
```

Vérifications

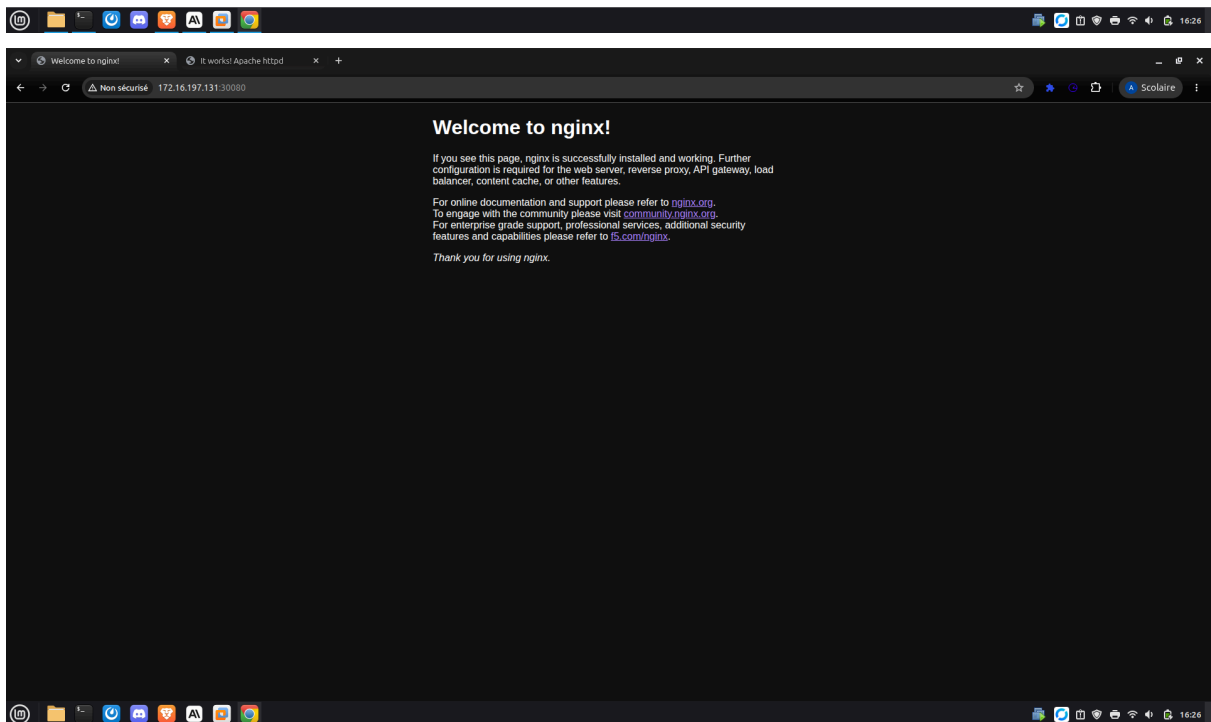
État des pods : *kubect! get pods*

Services et NodePorts : *kubect! get services*

```
root@kubes-01:~/deployments# kubect! get services
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
apache-svc NodePort     10.43.40.199 <none>        80:30081/TCP     3h22m
kubernetes ClusterIP    10.43.0.1     <none>        443/TCP          22h
mariadb-svc NodePort     10.43.171.230 <none>        3306:30306/TCP   3h22m
nginx-svc  NodePort     10.43.206.104 <none>        80:30080/TCP     3h17m
```

Test d'accès depuis le navigateur

Application	URL kubes-01	URL kubes-02	URL kubes-03
Nginx	http://172.16.197.131:30080	http://172.16.197.132:30080	http://172.16.197.133:30080
Apache	http://172.16.197.131:30081	http://172.16.197.132:30081	http://172.16.197.133:30081



Test de connexion MariaDB

Entrer dans le pod MariaDB:

```
kubectl exec -it $(kubectl get pod -l app=mariadb -o  
jsonpath='{.items[0].metadata.name}') -- bash
```

Une fois dans le pod : *mariadb -u root -prootpassword*

```

root@mariadb-c68cf7b8c-87bnx:/# mariadb -u root -prootpassword
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 12.3.2-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```

Job 03 .Création du cluster K3S (master + workers)

Transformation des 3 instances K3S standalone en un vrai cluster :

- **kubes-01** → master (control-plane)
- **kubes-02** → worker
- **kubes-03** → worker

1.Récupérer le token sur le master (kubes-01)

Le token K3S permet aux workers de s'authentifier auprès du master.

cat /var/lib/rancher/k3s/server/node-token

```

root@kubes-01:~/deployments# cat /var/lib/rancher/k3s/server/node-token
K102ebd2d88ba35ae26af6d07b0d3e2dfac466befd38d777123b9850c71e64fc347::server:96fc708e35c9f466fb6d4a23a9bf6f37

```

2. Désinstaller K3S standalone sur les workers

K3S standalone doit être désinstallé avant de rejoindre le cluster.

Sur kubes-02 ET kubes-03 : */usr/local/bin/k3s-uninstall.sh*

Vérifier la désinstallation : *systemctl status k3s*

3.Rejoindre le cluster

Sur kubes-02 :

```

curl -sfL https://get.k3s.io | \
  K3S_URL=https://172.16.227.10:6443 \
  K3S_TOKEN=<TOKEN_DU_MASTER> \
  INSTALL_K3S_EXEC="--node-ip=172.16.227.11 --flannel-iface=ens34" sh -

```

Sur kubes-03

```

curl -sfL https://get.k3s.io | \
  K3S_URL=https://172.16.227.10:6443 \
  K3S_TOKEN=<TOKEN_DU_MASTER> \
  INSTALL_K3S_EXEC="--node-ip=172.16.227.12 --flannel-iface=ens34" sh -

```

K3S_URL pointe vers l'API server du master sur le port 6443 via l'IP Host-Only.

```
root@kubes-01:~# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE           NOMINATED NODE   READINESS GATES
apache-84647dcc49-sk7cp             1/1     Running   1 (25m ago)  18h   10.42.0.20    kubes-01.local <none>           <none>
mariadb-c68cf7b8c-2xx25             1/1     Running   1 (25m ago)  18h   10.42.0.24    kubes-01.local <none>           <none>
nginx-59f86b59ff-m9t9j             1/1     Running   1 (25m ago)  18h   10.42.0.21    kubes-01.local <none>           <none>
```

4. Vérifier le cluster

Depuis kubes-01 uniquement : *kubectl get nodes*

```
^Croot@kubes-01:~/deployments# kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
kubes-01.local      Ready    control-plane  21h   v1.35.5+k3s1
kubes-02.local      Ready    <none>      176m   v1.35.5+k3s1
kubes-03.local      Ready    <none>      174m   v1.35.5+k3s1
```

5. Vérifier les applications après clustering : *kubectl get pods -o wide*

```
root@kubes-01:~/deployments# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE           NOMINATED NODE   READINESS GATES
apache-84647dcc49-b65cp             1/1     Running   0          102m   10.42.3.6     kubes-03.local <none>           <none>
apache-84647dcc49-jd9rk             1/1     Running   0          4h1m   10.42.0.25    kubes-01.local <none>           <none>
apache-84647dcc49-vzwq6             1/1     Running   0          4h1m   10.42.3.3     kubes-03.local <none>           <none>
mariadb-c68cf7b8c-87bnx             1/1     Running   0          4h1m   10.42.3.4     kubes-03.local <none>           <none>
mariadb-c68cf7b8c-c698t             1/1     Running   0          4h1m   10.42.0.26    kubes-01.local <none>           <none>
mariadb-c68cf7b8c-xvn75             1/1     Running   0          102m   10.42.3.8     kubes-03.local <none>           <none>
nginx-59f86b59ff-9qdps             1/1     Running   0          3h55m  10.42.0.27    kubes-01.local <none>           <none>
nginx-59f86b59ff-hlldv             1/1     Running   0          102m   10.42.3.7     kubes-03.local <none>           <none>
nginx-59f86b59ff-rwm8n             1/1     Running   0          3h55m  10.42.3.5     kubes-03.local <none>           <none>
```

Les pods tournent uniquement sur kubes-01 car ils ont été déployés avant le clustering. K3S ne redistribue pas automatiquement les pods existants sur les nouveaux workers.

Job 04 .Haute Disponibilité (HA) avec replicas

Mise en place de la haute disponibilité en redéployant toutes les applications avec **replicas: 3** afin de répartir les pods sur les 3 nodes du cluster. Test de la HA en simulant la perte d'un worker.

1. Suppression des déploiements existants

cd ~/deployments

kubectl delete -f nginx.yaml

kubectl delete -f apache.yaml

kubectl delete -f mariadb.yaml

```
root@kubes-01:~/deployments# kubectl delete -f nginx.yaml
deployment.apps "nginx" deleted from default namespace
service "nginx-svc" deleted from default namespace
root@kubes-01:~/deployments# kubectl delete -f apache.yaml
deployment.apps "apache" deleted from default namespace
service "apache-svc" deleted from default namespace
root@kubes-01:~/deployments# kubectl delete -f mariadb.yaml
deployment.apps "mariadb" deleted from default namespace
service "mariadb-svc" deleted from default namespace
```

Vérifier que tout est supprimé : *kubectl get pods*

```
root@kubes-01:~/deployments# kubectl get pods
No resources found in default namespace.
```

2. Mise à jour des fichiers YAML (replicas: 3)

Seul le champ *replicas* change

nginx.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  spec:
    replicas: 3
    selector:
      matchLabels:
        app: nginx
    template:
      metadata:
        labels:
          app: nginx
      spec:
        containers:
          - name: nginx
            image: nginx:latest
            ports:
              - containerPort: 80
---
apiVersion: v1
kind: Service

metadata:
  name: nginx-svc

spec:
```

```
type: NodePort
selector:
  app: nginx
ports:
- port: 80
  nodePort: 30080
```

apache.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache
spec:
  replicas: 3
  selector:
    matchLabels:
      app: apache
  template:
    metadata:
      labels:
        app: apache
    spec:
      containers:
      - name: apache
        image: httpd:latest
```

```
    ports:
      - containerPort: 80
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: apache-svc
```

```
spec:
```

```
  type: NodePort
```

```
  selector:
```

```
    app: apache
```

```
  ports:
```

```
    - port: 80
```

```
      nodePort: 30081
```

mariadb.yaml

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: mariadb
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: mariadb
```

template:

metadata:

labels:

app: mariadb

spec:

containers:

- name: mariadb

image: mariadb:latest

env:

- name: MYSQL_ROOT_PASSWORD

value: "rootpassword"

ports:

- containerPort: 3306

apiVersion: v1

kind: Service

metadata:

name: mariadb-svc

spec:

type: NodePort

selector:

app: mariadb

ports:

- port: 3306

nodePort: 30306

3.Redéploiement avec HA

kubectl apply -f nginx.yaml

kubectl apply -f apache.yaml

kubectl apply -f mariadb.yaml

```
root@kubes-01:~/deployments# kubectl apply -f nginx.yaml
deployment.apps/nginx created
service/nginx-svc created
root@kubes-01:~/deployments# kubectl apply -f apache.yaml
deployment.apps/apache unchanged
service/apache-svc unchanged
root@kubes-01:~/deployments# kubectl apply -f mariadb.yaml
error: the path "mariadb.yaml" does not exist
root@kubes-01:~/deployments# kubectl apply -f mariadb.yaml
deployment.apps/mariadb unchanged
service/mariadb-svc unchanged
```

Suivre le démarrage : *kubectl get pods -o wide -w*

```
root@kubes-01:~/deployments# kubectl get pods -o wide -w
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE              NOMINATED NODE   READINESS GATES
apache-84647dcc49-h9nv2             1/1     Running   0           6m46s  10.42.1.3     kubes-02.local    <none>            <none>
apache-84647dcc49-jd9rk             1/1     Running   0           6m46s  10.42.0.25    kubes-01.local    <none>            <none>
apache-84647dcc49-vzwq6             1/1     Running   0           6m46s  10.42.3.3     kubes-03.local    <none>            <none>
mariadb-c68cf7b8c-87bnx            1/1     Running   0           6m45s  10.42.3.4     kubes-03.local    <none>            <none>
mariadb-c68cf7b8c-c698t            1/1     Running   0           6m45s  10.42.0.26    kubes-01.local    <none>            <none>
mariadb-c68cf7b8c-hff7v            1/1     Running   0           6m45s  10.42.1.4     kubes-02.local    <none>            <none>
nginx-59f86b59ff-2tdgn             1/1     Running   0           69s    10.42.1.5     kubes-02.local    <none>            <none>
nginx-59f86b59ff-9qdp5             1/1     Running   0           69s    10.42.0.27    kubes-01.local    <none>            <none>
nginx-59f86b59ff-rwm8n             1/1     Running   0           69s    10.42.3.5     kubes-03.local    <none>            <none>
```

4.Test de la Haute Disponibilité

4.1 Arrêt du worker kubes-02

Sur kubes-02 : *systemctl stop k3s-agent*

4.2 Observation depuis kubes-01

Vérifier l'état des nodes : *kubectl get nodes*

```
^Croot@kubes-01:~/deployments# kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
kubes-01.local      Ready     control-plane  21h   v1.35.5+k3s1
kubes-02.local      NotReady  <none>        163m   v1.35.5+k3s1
kubes-03.local      Ready     <none>        161m   v1.35.5+k3s1
```

4.3 Rescheduling automatique des pods : *kubectl get pods -o wide -w*

Après environ 2-5 minutes, Kubernetes détecte que kubes-02 est indisponible et redistribue automatiquement ses pods sur kubes-01 et kubes-03.

5. Remise en service de kubes-02

Sur kubes-02 : `systemctl start k3s-agent`

Sur kubes-01, vérifier : `kubectl get nodes`

```
root@kubes-01:~/deployments# kubectl get pods -o wide -w
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
apache-84647dcc49-h9nv2             1/1    Running   0           138m  10.42.1.3       kubes-02.local   <none>           <none>
apache-84647dcc49-jd9rk             1/1    Running   0           138m  10.42.0.25      kubes-01.local   <none>           <none>
apache-84647dcc49-vzwq6             1/1    Running   0           138m  10.42.3.3       kubes-03.local   <none>           <none>
mariadb-c68cf7b8c-87bnx             1/1    Running   0           138m  10.42.3.4       kubes-03.local   <none>           <none>
mariadb-c68cf7b8c-c698t            1/1    Running   0           138m  10.42.0.26      kubes-01.local   <none>           <none>
mariadb-c68cf7b8c-hff7v            1/1    Running   0           138m  10.42.1.4       kubes-02.local   <none>           <none>
nginx-59f86b59ff-2tdgn             1/1    Running   0           132m  10.42.1.5       kubes-02.local   <none>           <none>
nginx-59f86b59ff-9qdp5             1/1    Running   0           132m  10.42.0.27      kubes-01.local   <none>           <none>
nginx-59f86b59ff-rwm8n             1/1    Running   0           132m  10.42.3.5       kubes-03.local   <none>           <none>

root@kubes-01:~/deployments# kubectl get pods -o wide -w
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
apache-84647dcc49-b65cp             1/1    Running   0           6m52s  10.42.3.6       kubes-03.local   <none>           <none>
apache-84647dcc49-h9nv2             1/1    Terminating 0       145m  10.42.1.3       kubes-02.local   <none>           <none>
apache-84647dcc49-jd9rk             1/1    Running   0           145m  10.42.0.25      kubes-01.local   <none>           <none>
apache-84647dcc49-vzwq6             1/1    Running   0           145m  10.42.3.3       kubes-03.local   <none>           <none>
mariadb-c68cf7b8c-87bnx             1/1    Running   0           145m  10.42.3.4       kubes-03.local   <none>           <none>
mariadb-c68cf7b8c-c698t            1/1    Running   0           145m  10.42.0.26      kubes-01.local   <none>           <none>
mariadb-c68cf7b8c-hff7v            1/1    Terminating 0       145m  10.42.1.4       kubes-02.local   <none>           <none>
mariadb-c68cf7b8c-xvn75            1/1    Running   0           6m52s  10.42.3.8       kubes-03.local   <none>           <none>
nginx-59f86b59ff-2tdgn             1/1    Terminating 0       139m  10.42.1.5       kubes-02.local   <none>           <none>
nginx-59f86b59ff-9qdp5             1/1    Running   0           139m  10.42.0.27      kubes-01.local   <none>           <none>
nginx-59f86b59ff-hlldv             1/1    Running   0           6m52s  10.42.3.7       kubes-03.local   <none>           <none>
nginx-59f86b59ff-rwm8n             1/1    Running   0           139m  10.42.3.5       kubes-03.local   <none>           <none>
nginx-59f86b59ff-2tdgn             1/1    Terminating 0       141m  10.42.1.5       kubes-02.local   <none>           <none>
mariadb-c68cf7b8c-hff7v            1/1    Terminating 0       146m  10.42.1.4       kubes-02.local   <none>           <none>
apache-84647dcc49-h9nv2             1/1    Terminating 0       146m  10.42.1.3       kubes-02.local   <none>           <none>
nginx-59f86b59ff-2tdgn             0/1    Completed   0       141m  10.42.1.5       kubes-02.local   <none>           <none>
mariadb-c68cf7b8c-hff7v            0/1    Completed   0       146m  10.42.1.4       kubes-02.local   <none>           <none>
apache-84647dcc49-h9nv2             0/1    Completed   0       146m  10.42.1.3       kubes-02.local   <none>           <none>
nginx-59f86b59ff-2tdgn             0/1    Completed   0       141m  10.42.1.5       kubes-02.local   <none>           <none>
nginx-59f86b59ff-2tdgn             0/1    Completed   0       141m  10.42.1.5       kubes-02.local   <none>           <none>
mariadb-c68cf7b8c-hff7v            0/1    Completed   0       146m  10.42.1.4       kubes-02.local   <none>           <none>
mariadb-c68cf7b8c-hff7v            0/1    Completed   0       146m  10.42.1.4       kubes-02.local   <none>           <none>
apache-84647dcc49-h9nv2             0/1    Completed   0       146m  10.42.1.3       kubes-02.local   <none>           <none>
apache-84647dcc49-h9nv2             0/1    Completed   0       146m  10.42.1.3       kubes-02.local   <none>           <none>
```

```
^Croot@kubes-01:~/deployments# kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
kubes-01.local      Ready     control-plane 21h   v1.35.5+k3s1
kubes-02.local      Ready     <none>      176m  v1.35.5+k3s1
kubes-03.local      Ready     <none>      174m  v1.35.5+k3s1
```

Job 05. Volumes Persistants (PV/PVC)

Les volumes persistants permettent aux applications de conserver leurs données même après le redémarrage ou la recreation des pods.

Mise en place pour :

- **Nginx** → stockage des fichiers web (`/usr/share/nginx/html`)

MariaDB → stockage des données de la base (`/var/lib/mariadb`) K3S inclut un provisionneur automatique `local-path` qui crée ses propres volumes si aucun `storageClassName` n'est spécifié. Il faut donc toujours préciser `storageClassName: manual` pour lier explicitement PV et PVC.

Fichiers YAML

nginx-pv.yaml

```
apiVersion: v1

kind: PersistentVolume

metadata:

  name: nginx-pv

spec:

  capacity:

    storage: 1Gi

  accessModes:

    - ReadWriteOnce

  storageClassName: manual

  hostPath:

    path: /data/nginx

  persistentVolumeReclaimPolicy: Retain
```

```
apiVersion: v1

kind: PersistentVolumeClaim

metadata:

  name: nginx-pvc

spec:

  accessModes:

    - ReadWriteOnce

  storageClassName: manual

  resources:
```

requests:

storage: 1Gi

mariadb-pv.yaml

apiVersion: v1

kind: PersistentVolume

metadata:

name: mariadb-pv

spec:

capacity:

storage: 5Gi

accessModes:

- ReadWriteOnce

storageClassName: manual

hostPath:

path: /data/mariadb

persistentVolumeReclaimPolicy: Retain

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

name: mariadb-pvc

spec:

accessModes:

- ReadWriteOnce

storageClassName: manual

resources:

requests:

storage: 5Gi

nginx.yaml (avec volume)

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx

spec:

replicas: 3

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx:latest

ports:

- containerPort: 80

```
    volumeMounts:
      - name: nginx-data
        mountPath: /usr/share/nginx/html

    volumes:
      - name: nginx-data
        persistentVolumeClaim:
          claimName: nginx-pvc
---

apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      nodePort: 30080
```

mariadb.yaml (avec volume)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb
```

spec:

replicas: 3

selector:

matchLabels:

app: mariadb

template:

metadata:

labels:

app: mariadb

spec:

containers:

- name: mariadb

image: mariadb:latest

env:

- name: MYSQL_ROOT_PASSWORD

value: "rootpassword"

ports:

- containerPort: 3306

volumeMounts:

- name: mariadb-data

mountPath: /var/lib/mysql

volumes:

- name: mariadb-data

persistentVolumeClaim:

claimName: mariadb-pvc

apiVersion: v1

kind: Service

metadata:

name: mariadb-svc

spec:

type: NodePort

selector:

app: mariadb

ports:

- port: 3306

nodePort: 30306

Déploiement

Appliquer les PV/PVC en premier

kubectl apply -f nginx-pv.yaml

kubectl apply -f mariadb-pv.yaml

Vérifier que les PVC sont Bound avant de déployer

kubectl get pv

kubectl get pvc

Déployer les applications

kubectl apply -f nginx.yaml

kubectl apply -f mariadb.yaml

kubectl apply -f apache.yaml

Vérifications

État des PV et PVC:

kubectl get pv

kubectl get pvc

```
root@kubes-01:~/deployments# kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS	VOLUMEATTRIBUTESCLASS	REASON	AGE
mariadb-pv	5Gi	RWO	Retain	Bound	default/mariadb-pvc	manual	<unset>		8s
nginx-pv	1Gi	RWO	Retain	Bound	default/nginx-pvc	manual	<unset>		19s

```
root@kubes-01:~/deployments# kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
mariadb-pvc	Bound	mariadb-pv	5Gi	RWO	manual	<unset>	18s
nginx-pvc	Bound	nginx-pv	1Gi	RWO	manual	<unset>	29s

État des pods : *kubectl get pods -o wide*

```
root@kubes-01:~/deployment# kubectl get pods -o wide -w
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
apache-84647dcc49-b65cp	1/1	Running	0	167m	10.42.3.6	kubes-03.local	<none>	<none>
apache-84647dcc49-jd9rk	1/1	Running	0	5h5m	10.42.0.25	kubes-01.local	<none>	<none>
apache-84647dcc49-vzwq6	1/1	Running	0	5h5m	10.42.3.3	kubes-03.local	<none>	<none>
mariadb-6dbb754947-5ntlz	1/1	Running	0	48s	10.42.3.15	kubes-03.local	<none>	<none>
mariadb-6dbb754947-n878m	1/1	Running	0	48s	10.42.0.29	kubes-01.local	<none>	<none>
mariadb-6dbb754947-v5jz7	1/1	Running	0	48s	10.42.1.12	kubes-02.local	<none>	<none>
nginx-5cbff5cc78-6gvvcg	1/1	Running	0	59s	10.42.1.11	kubes-02.local	<none>	<none>
nginx-5cbff5cc78-b268s	1/1	Running	0	59s	10.42.0.28	kubes-01.local	<none>	<none>
nginx-5cbff5cc78-ntg4x	1/1	Running	0	59s	10.42.3.14	kubes-03.local	<none>	<none>

Problème rencontré et solution

Problème

K3S dispose d'un provisionneur automatique *local-path* qui intercepte les PVC sans *storageClassName* et crée ses propres volumes dynamiques, laissant les PV manuels en statut *Available*.

Solution

Ajouter *storageClassName: manual* à la fois dans le PV et dans le PVC pour forcer la liaison explicite entre les deux.

Dans le PV ET dans le PVC : *storageClassName: manual*

Job 06. Gestion de la configuration avec ConfigMaps

Les ConfigMaps permettent de séparer la configuration du code applicatif. Ils sont montés comme des fichiers dans les pods, permettant de modifier la configuration sans reconstruire les images Docker.

Mise en place pour :

- **Nginx** → page d'accueil personnalisée (*/usr/share/nginx/html/index.html*)
- **Apache** → page d'accueil personnalisée (*/usr/local/apache2/htdocs/index.html*)

Fichiers YAML

nginx-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  index.html: |
    <!DOCTYPE html>
    <html>
    <head><title>Nginx K3S</title></head>
    <body>
      <h1>Bienvenue sur Nginx</h1>
      <p>Deploye sur le cluster K3S - La Plateforme</p>
    </body>
    </html>
```

apache-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: apache-config
data:
  index.html: |
    <!DOCTYPE html>
    <html>
    <head><title>Apache K3S</title></head>
    <body>
      <h1>Bienvenue sur Apache</h1>
      <p>Deploye sur le cluster K3S - La Plateforme</p>
    </body>
    </html>
```

nginx.yaml (avec ConfigMap)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
```

```
  matchLabels:
    app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
        volumeMounts:
        - name: nginx-data
          mountPath: /usr/share/nginx/html
        - name: nginx-config
          mountPath: /usr/share/nginx/html/index.html
          subPath: index.html
      volumes:
      - name: nginx-data
        persistentVolumeClaim:
          claimName: nginx-pvc
      - name: nginx-config
        configMap:
          name: nginx-config
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
```

spec:

type: NodePort

selector:

app: nginx

ports:

- port: 80

nodePort: 30080

apache.yaml (avec ConfigMap)

apiVersion: apps/v1

kind: Deployment

metadata:

name: apache

spec:

replicas: 3

selector:

matchLabels:

app: apache

template:

metadata:

labels:

app: apache

spec:

containers:

- name: apache

image: httpd:latest

ports:

- containerPort: 80

volumeMounts:

- name: apache-config

mountPath: /usr/local/apache2/htdocs/index.html

subPath: index.html

volumes:

- name: apache-config

configMap:

name: apache-config

apiVersion: v1

kind: Service

metadata:

name: apache-svc

spec:

type: NodePort

selector:

app: apache

ports:

- port: 80

nodePort: 30081

Déploiement

Appliquer les ConfigMaps

```
kubectl apply -f nginx-configmap.yaml
```

```
kubectl apply -f apache-configmap.yaml
```

Redéployer les applications

```
kubectl apply -f nginx.yaml
```

```
kubectl apply -f apache.yaml
```

Vérifications

Lister les ConfigMaps : `kubectl get configmaps`

```
root@kubes-01:~/deployments# kubectl get configmaps
NAME              DATA   AGE
apache-config     1       27s
kube-root-ca.crt  1       24h
nginx-config      1       35s
```

Vérifier le contenu monté dans le pod Apache

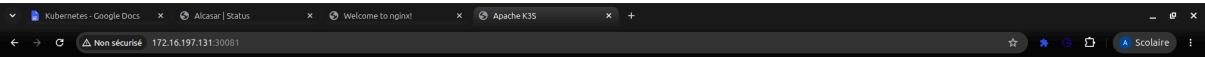
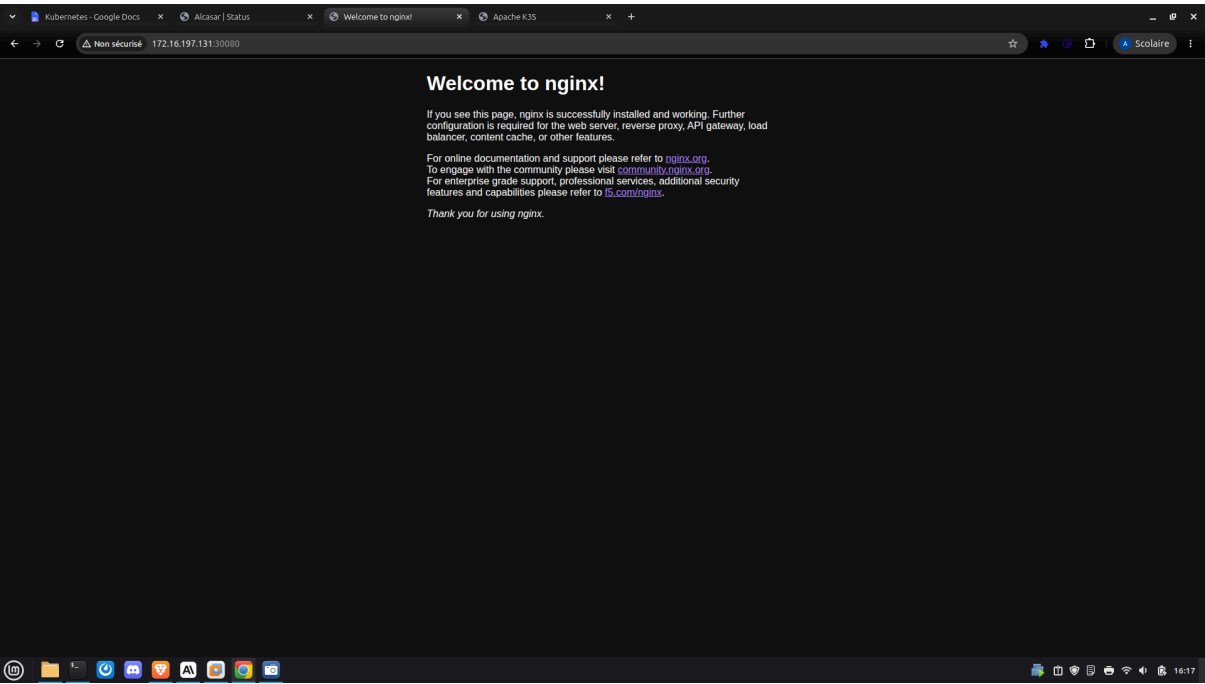
```
kubectl exec -it $(kubectl get pod -l app=apache \
-o jsonpath='{.items[0].metadata.name}') -- \
```

```
cat /usr/local/apache2/htdocs/index.html
```

```
root@kubes-01:~/deployments# kubectl exec -it $(kubectl get pod -l app=apache -o jsonpath='{.items[0].metadata.name}') -- bash
root@apache-8f5d5f5c-9x97c:/usr/local/apache2# find / -name "index.html" 2>/dev/null
/usr/local/apache2/htdocs/index.html
root@apache-8f5d5f5c-9x97c:/usr/local/apache2# cat /usr/local/apache2/htdocs/index.html
<!DOCTYPE html>
<html>
<head><title>Apache K3S</title></head>
<body>
  <h1>Bienvenue sur Apache</h1>
  <p>Deploye sur le cluster K3S - La Plateforme</p>
</body>
</html>
```

```
root@kubes-01:~/deployments# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
apache-84647dcc49-jd9rk             0/1     Completed 0           5h22m 10.42.0.25      kubes-01.local   <none>            <none>
apache-8f5d5f5c-9x97c              1/1     Running   0           7s    10.42.0.31      kubes-01.local   <none>            <none>
apache-8f5d5f5c-gbmlk              1/1     Running   0          11s    10.42.3.17      kubes-03.local   <none>            <none>
apache-8f5d5f5c-r8ssg              1/1     Running   0          19s    10.42.1.14      kubes-02.local   <none>            <none>
mariadb-6dbb754947-5ntlz           1/1     Running   0          17m    10.42.3.15      kubes-03.local   <none>            <none>
mariadb-6dbb754947-n878m           1/1     Running   0          17m    10.42.0.29      kubes-01.local   <none>            <none>
mariadb-6dbb754947-v5jz7           1/1     Running   0          17m    10.42.1.12      kubes-02.local   <none>            <none>
nginx-77fbf9dc56-8ct7c             1/1     Running   0          25s    10.42.3.16      kubes-03.local   <none>            <none>
nginx-77fbf9dc56-lp5lh             1/1     Running   0          29s    10.42.1.13      kubes-02.local   <none>            <none>
nginx-77fbf9dc56-pngv7             1/1     Running   0          20s    10.42.0.30      kubes-01.local   <none>            <none>
```

Test depuis le navigateur



Problème rencontré et solution

Problème

Apache affichait toujours "It works!" malgré le ConfigMap correctement appliqué.

Diagnostic

Vérification du chemin réel dans le conteneur

```
kubectrl exec -it <pod-apache> -- find / -name "index.html" 2>/dev/null  
  
# /usr/local/apache2/htdocs/index.html
```

Vérification du contenu : fichier bien présent et correct

```
kubectrl exec -it <pod-apache> -- cat /usr/local/apache2/htdocs/index.html
```

et affichait bien "Bienvenue sur Apache"

Solution

Le fichier était correctement monté ; c'était un problème de **cache navigateur** qui a été résolu avec **Ctrl+Shift+R** (rechargement forcé sans cache).

Job 07.Données sensibles avec Secrets (MariaDB)

Les Secrets Kubernetes permettent de stocker des données sensibles (mots de passe, clés API, certificats TLS) de façon chiffrée, sans les exposer en clair dans les fichiers YAML.

Mise en place pour :

- **MariaDB** : mot de passe root stocké dans un Secret

1.Encoder le mot de passe en base64

```
echo -n 'rootpassword' | base64
```

```
# cm9vdHBhc3N3b3Jk
```

L'option **-n** évite d'encoder le caractère de fin de ligne **\n**.

2. Créer le Secret

mariadb-secret.yaml

apiVersion: v1

kind: Secret

metadata:

name: mariadb-secret

type: Opaque

data:

MYSQL_ROOT_PASSWORD: cm9vdHBhc3N3b3Jk

kubectl apply -f mariadb-secret.yaml

3.Mettre à jour mariadb.yaml

Remplacement de la valeur en clair par une référence au Secret :

Avant (mot de passe en clair) :

env:

- name: MYSQL_ROOT_PASSWORD

value: "rootpassword"

Après (référence au Secret) :

env:

- name: MYSQL_ROOT_PASSWORD

valueFrom:

secretKeyRef:

name: mariadb-secret

key: MYSQL_ROOT_PASSWORD

mariadb.yaml complet

apiVersion: apps/v1

kind: Deployment

metadata:

name: mariadb

spec:

replicas: 3

selector:

matchLabels:

app: mariadb

template:

metadata:

labels:

app: mariadb

spec:

containers:

- name: mariadb

image: mariadb:latest

env:

- name: MYSQL_ROOT_PASSWORD

valueFrom:

secretKeyRef:

name: mariadb-secret

key: MYSQL_ROOT_PASSWORD

```
    ports:
      - containerPort: 3306

    volumeMounts:
      - name: mariadb-data
        mountPath: /var/lib/mysql

    volumes:
      - name: mariadb-data
        persistentVolumeClaim:
          claimName: mariadb-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mariadb-svc
spec:
  type: NodePort
  selector:
    app: mariadb
  ports:
    - port: 3306
      nodePort: 30306
```

kubectl apply -f mariadb.yaml

Vérifications

Lister les Secrets : *kubectl get secrets*

```
root@kubes-01:~/deployments# kubectl get secrets
NAME          TYPE      DATA  AGE
mariadb-secret  Opaque    1       2m22s
```

Inspecter le Secret : *kubectl describe secret mariadb-secret*

```
root@kubes-01:~/deployments# kubectl describe secret mariadb-secret
Name:          mariadb-secret
Namespace:     default
Labels:        <none>
Annotations:    <none>

Type:  Opaque

Data
====
MYSQL_ROOT_PASSWORD:  12 bytes
```

La valeur n'est jamais affichée en clair — Kubernetes affiche uniquement la taille en bytes (12 bytes = longueur de "rootpassword").

Test de connexion MariaDB avec le Secret

```
kubectl exec -it $(kubectl get pod -l app=mariadb \
-o jsonpath='{.items[0].metadata.name}') -- bash
```

```
mysql -u root -prootpassword
```

```
root@kubes-01:~/deployments# kubectl exec -it $(kubectl get pod -l app=mariadb \
-o jsonpath='{.items[0].metadata.name}') -- bash
root@mariadb-5b9587b47b-4hd9t:/# mysql -u root -prootpassword
bash: mysql: command not found
root@mariadb-5b9587b47b-4hd9t:/# mariadb -u root -prootpassword
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 12.3.2-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> exit
Bye
root@mariadb-5b9587b47b-4hd9t:/# exit
exit
```

Job 08 .Règles RBAC (Role Based Access Control)

Le RBAC permet de contrôler finement qui peut faire quoi sur quelles ressources du cluster. C'est un mécanisme de sécurité fondamental en production pour appliquer le principe du moindre privilège.

Principe du moindre privilège: Ne donner que les droits strictement nécessaires. Un ServiceAccount ne doit jamais avoir plus de permissions que ce dont il a besoin.

Fichier YAML

rbac.yaml

ServiceAccount

apiVersion: v1

kind: ServiceAccount

metadata:

name: app-sa

namespace: default

Role — lecture seule sur les pods

apiVersion: rbac.authorization.k8s.io/v1

kind: Role

metadata:

namespace: default

name: pod-reader

rules:

- apiGroups: [""]

resources: ["pods"]

verbs: ["get", "list", "watch"]

RoleBinding — lier le Role au ServiceAccount

apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding

metadata:

name: read-pods

namespace: default

subjects:

- kind: ServiceAccount

name: app-sa

namespace: default

roleRef:

kind: Role

name: pod-reader

apiGroup: rbac.authorization.k8s.io

Déploiement : *kubectl apply -f rbac.yaml*

Vérifications

Lister les ressources RBAC créées:

kubectl get serviceaccounts

kubectl get roles

kubectl get rolebindings

```
root@kubes-01:~/deployments# kubectl get serviceaccounts
NAME      AGE
app-sa    20s
default   40h
root@kubes-01:~/deployments# kubectl get roles
NAME          CREATED AT
pod-reader    2026-06-10T06:30:08Z
root@kubes-01:~/deployments# kubectl get rolebindings
NAME          ROLE           AGE
read-pods     Role/pod-reader 44s
```

Décrire le Role : *kubectl describe role pod-reader*

```
root@kubes-01:~/deployments# kubectl describe role pod-reader
Name:         pod-reader
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -
  pods      []                 []              [get list watch]
```

Test des permissions

➤ *app-sa PEUT lister les pods:*
*kubectl auth can-i list pods *
--as=system:serviceaccount:default:app-sa
⇒ *yes*

➤ *app-sa NE PEUT PAS supprimer les pods:*
*kubectl auth can-i delete pods *
--as=system:serviceaccount:default:app-sa
⇒ *no*

➤ *app-sa NE PEUT PAS lister les services :*
*kubectl auth can-i list services *
--as=system:serviceaccount:default:app-sa
⇒ *no*

➤ *app-sa NE PEUT PAS créer des deployments :*
*kubectl auth can-i create deployments *
--as=system:serviceaccount:default:app-sa
⇒ *no*

Job 09 .Helm : Le gestionnaire de paquets Kubernetes

Pour déployer une application sur Kubernetes manuellement, il faut créer et gérer plusieurs fichiers YAML séparément. Par exemple pour Nginx seul :

nginx.yaml → Deployment + Service

nginx-pv.yaml → PersistentVolume + PersistentVolumeClaim

nginx-configmap.yaml → ConfigMap

Soit 3 fichiers, des dizaines de lignes YAML, tout géré à la main. Pour une application complexe comme WordPress (qui nécessite aussi MySQL, des volumes, des secrets, des ingress...), cela peut représenter 10-15 fichiers.

La solution Helm

*Helm regroupe tous ces fichiers dans un **Chart** — un paquet réutilisable et versionné. C'est l'équivalent de **apt install** sur Debian : au lieu de compiler un logiciel à la main, tu l'installes en une commande.*

1.Installation de Helm

*curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash*

Vérifier l'installation : helm version

```
root@kubes-01:~/deployments# helm version
version.BuildInfo{Version:"v3.21.0", GitCommit:"e0878d41b711792be60777fd65ad23a101e6b85f", GitTreeState:"clean", GoVersion:"go1.25.10"}
```

2.Configuration pour K3S

K3S stocke sa config `kubectrl` dans un chemin différent de Kubernetes standard. Il faut indiquer à Helm où la trouver: *export KUBECONFIG=/etc/rancher/k3s/k3s.yaml*

Rendre permanent au redémarrage:

*echo "export KUBECONFIG=/etc/rancher/k3s/k3s.yaml" >> ~/.bashrc
source ~/.bashrc*

3. Ajouter les repositories

Les repositories sont des catalogues de charts, comme les sources apt sur Debian.

Ajouter le repo Bitnami :

helm repo add bitnami https://charts.bitnami.com/bitnami

Mettre à jour le cache local des repos : *helm repo update*

Vérifier les repos disponibles : *helm repo list*

```
root@kubes-01:~/deployments# helm repo list
NAME      URL
bitnami   https://charts.bitnami.com/bitnami
```

4.Rechercher et explorer un chart

Rechercher nginx dans tous les repos : *helm search repo nginx*

Voir les détails d'un chart : *helm show chart bitnami/nginx*

Voir toutes les options de configuration disponibles : *helm show values bitnami/nginx*

```
root@kubes-01:~/deployments# helm search repo nginx
NAME                CHART VERSION  APP VERSION  DESCRIPTION
bitnami/nginx        25.0.3         1.31.1       NGINX Open Source is a web server that can be a...
bitnami/nginx-ingress-controller 12.0.7         1.13.1       NGINX Ingress Controller is an Ingress controll...
bitnami/nginx-intel  2.1.15         0.4.9        DEPRECATED NGINX Open Source for Intel is a lig...
```

5.Installer une application avec une simple release

Installer nginx avec le nom de release "mon-nginx":

helm install mon-nginx bitnami/nginx

Ce que Helm fait en coulisses en une seule commande :

- Télécharge le chart bitnami/nginx
- Crée le Deployment
- Crée le Service
- Crée les ConfigMaps nécessaires
- Gère les dépendances

Vérifier la release installée : *helm list*

Vérifier les pods créés:

kubectl get pods

kubectl get services

```
root@kubes-01:~/deployments# helm list
NAME      NAMESPACE  REVISION  UPDATED                               STATUS  CHART          APP VERSION
mon-nginx default     1         2026-06-10 08:53:04.948248459 +0200 CEST  deployed  nginx-25.0.3   1.31.1
```

6.Personnaliser avec values.yaml

Le fichier *values.yaml* permet de surcharger la configuration par défaut du chart sans modifier le chart lui-même.

cat > values-nginx.yaml << 'EOF'

replicaCount: 3

service:

type: NodePort

nodePorts:

http: 30090

EOF

Installer avec valeurs personnalisées

```
helm install nginx-custom bitnami/nginx -f values-nginx.yaml
```

Vérifier — 3 replicas doivent être créés

```
helm list
```

```
kubectl get pods
```

```
kubectl get services
```

```
root@kubes-01:~/deployments# kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
apache-8f5d5f5c-9x97c	1/1	Running	1 (63m ago)	16h
apache-8f5d5f5c-gbmlk	1/1	Running	1 (62m ago)	16h
apache-8f5d5f5c-r8ssg	1/1	Running	1 (62m ago)	16h
mariadb-5b9587b47b-4hd9t	1/1	Running	0	45m
mariadb-5b9587b47b-d5b9x	1/1	Running	0	45m
mariadb-5b9587b47b-g5rd7	1/1	Running	0	45m
mon-nginx-77fd75878f-qqlhb	1/1	Running	0	117s
nginx-77fbf9dc56-8ct7c	1/1	Running	1 (62m ago)	16h
nginx-77fbf9dc56-lp5lh	1/1	Running	1 (62m ago)	16h
nginx-77fbf9dc56-pngv7	1/1	Running	1 (63m ago)	16h
nginx-custom-84bf4896d4-246vz	0/1	Init:0/1	0	20s
nginx-custom-84bf4896d4-lpnx9	0/1	Running	0	20s
nginx-custom-84bf4896d4-s6pb5	0/1	Init:0/1	0	20s

7. Mettre à jour une release (upgrade)

L'upgrade permet de modifier la configuration sans tout réinstaller. Helm garde l'historique de chaque modification.

Modifier le fichier values — passer à 2 replicas

```
cat > values-nginx.yaml << 'EOF'
```

```
replicaCount: 2
```

```
service:
```

```
  type: NodePort
```

```
  nodePorts:
```

```
    http: 30090
```

```
EOF
```

Mettre à jour la release:

```
helm upgrade nginx-custom bitnami/nginx -f values-nginx.yaml
```

Vérifier — *REVISION* doit passer à 2

```
helm list
```

```
kubectl get pods
```

```

root@kubes-01:~/deployments# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
apache-8f5d5f5c-9x97c              1/1     Running   1 (65m ago) 16h
apache-8f5d5f5c-gbmlk              1/1     Running   1 (64m ago) 16h
apache-8f5d5f5c-r8ssg              1/1     Running   1 (64m ago) 16h
mariadb-5b9587b47b-4hd9t           1/1     Running   0           47m
mariadb-5b9587b47b-d5b9x           1/1     Running   0           47m
mariadb-5b9587b47b-g5rd7           1/1     Running   0           47m
mon-nginx-77fd75878f-qqlhb         1/1     Running   0           3m24s
nginx-77fbf9dc56-8ct7c              1/1     Running   1 (64m ago) 16h
nginx-77fbf9dc56-lp5lh              1/1     Running   1 (64m ago) 16h
nginx-77fbf9dc56-pngv7              1/1     Running   1 (65m ago) 16h
nginx-custom-84bf4896d4-246vz       1/1     Running   0           107s
nginx-custom-84bf4896d4-lpnx9       1/1     Running   0           107s
root@kubes-01:~/deployments# helm history nginx-custom
REVISION    UPDATED                     STATUS      CHART          APP VERSION   DESCRIPTION
1           Wed Jun 10 08:54:43 2026   superseded  nginx-25.0.3   1.31.1        Install complete
2           Wed Jun 10 08:56:14 2026   deployed   nginx-25.0.3   1.31.1        Upgrade complete

```

8. Historique et Rollback

Voir tout l'historique des révisions : *helm history nginx-custom*

```

root@kubes-01:~/deployments# helm history nginx-custom
REVISION    UPDATED                     STATUS      CHART          APP VERSION   DESCRIPTION
1           Wed Jun 10 08:54:43 2026   superseded  nginx-25.0.3   1.31.1        Install complete
2           Wed Jun 10 08:56:14 2026   superseded  nginx-25.0.3   1.31.1        Upgrade complete
3           Wed Jun 10 08:57:17 2026   deployed   nginx-25.0.3   1.31.1        Rollback to 1

```

Rollback à la révision 1 (3 replicas) : *helm rollback nginx-custom 1*

```

root@kubes-01:~/deployments# helm rollback nginx-custom 1
Rollback was a success! Happy Helming!

```

9. Désinstaller une release

Désinstaller mon-nginx : *helm uninstall mon-nginx*

Vérifier mon-nginx ne doit plus apparaître

helm list

kubectl get pods

```

root@kubes-01:~/deployments# helm uninstall mon-nginx
release "mon-nginx" uninstalled
root@kubes-01:~/deployments# helm list
NAME                NAMESPACE      REVISION      UPDATED              STATUS      CHART          APP VERSION
nginx-custom        default         3             2026-06-10 08:57:17.2527 deployed    nginx-25.0.3   1.31.1
root@kubes-01:~/deployments# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
apache-8f5d5f5c-9x97c   1/1     Running   1 (66m ago)  16h
apache-8f5d5f5c-gbmlk   1/1     Running   1 (65m ago)  16h
apache-8f5d5f5c-r8ssg   1/1     Running   1 (65m ago)  16h
mariadb-5b9587b47b-4hd9t 1/1     Running   0           49m
mariadb-5b9587b47b-d5b9x 1/1     Running   0           49m
mariadb-5b9587b47b-g5rd7 1/1     Running   0           49m
nginx-77fbf9dc56-8ct7c   1/1     Running   1 (65m ago)  16h
nginx-77fbf9dc56-lp5lh   1/1     Running   1 (65m ago)  16h
nginx-77fbf9dc56-pngv7   1/1     Running   1 (66m ago)  16h
nginx-custom-84bf4896d4-246vz 1/1     Running   0           3m32s
nginx-custom-84bf4896d4-lpnx9 1/1     Running   0           3m32s
nginx-custom-84bf4896d4-tmnf5 1/1     Running   0           60s

```

Commandes Helm essentielles

```

helm repo add <nom> <url>      # Ajouter un repo

helm repo update                # Mettre à jour les repos

helm repo list                  # Lister les repos

helm search repo <terme>        # Rechercher un chart

helm install <release> <chart>   # Installer une release

helm install <release> <chart> -f values.yaml # Avec config custom

helm list                       # Lister les releases

helm status <release>           # État d'une release

helm upgrade <release> <chart>  # Mettre à jour

helm history <release>          # Historique des révisions

helm rollback <release> <n>     # Rollback à la révision n

helm uninstall <release>        # Désinstaller

helm show values <chart>        # Voir les options disponibles

```

Pour aller plus loin — Comparaison K3S, Docker et Docker Swarm

Dans le cadre de ce projet, nous avons utilisé K3S pour orchestrer nos conteneurs. Mais pourquoi K3S plutôt que Docker seul ou Docker Swarm ? Cette section compare les trois technologies, leurs avantages, inconvénients et cas d'usage.

Les trois technologies:

Docker — La brique de base

Docker est l'outil fondamental de la conteneurisation. Il permet de faire tourner des conteneurs sur **une seule machine**.

Lancer un conteneur Nginx avec Docker : `docker run -d -p 80:80 nginx`

Lancer plusieurs conteneurs avec Docker Compose : `docker-compose up -d`

Avantages :

- Très simple à apprendre et utiliser
- Parfait pour le développement local
- Léger et rapide à mettre en place
- Énorme communauté et documentation
- Base de toutes les autres technologies (Swarm, K3S utilisent Docker)

Inconvénients :

- Limité à une seule machine
- Pas de haute disponibilité
- Pas de rescheduling automatique si un conteneur tombe
- Pas de scaling automatique
- Pas adapté à la production à grande échelle

Docker Swarm — Le cluster Docker natif

Docker Swarm est le mode cluster intégré nativement dans Docker. Il permet de gérer plusieurs machines comme un seul cluster.

Initialiser un cluster Swarm : `docker swarm init --advertise-addr 192.x.x.x`

Ajouter un worker : `docker swarm join --token <TOKEN> 192.x.x.x:2377`

Déployer un service avec 3 replicas : `docker service create --replicas 3 -p 80:80 nginx`

Scaler un service : `docker service scale nginx=5`

Avantages :

- Intégré nativement dans Docker, aucune installation supplémentaire

- Très simple à configurer (quelques commandes)
- Utilise la même syntaxe que Docker Compose
- Suffisant pour des architectures simples
- Faible courbe d'apprentissage

Inconvénients :

- Beaucoup moins de fonctionnalités que Kubernetes
- Pas de gestion avancée du stockage (PV/PVC)
- Pas de RBAC natif
- Communauté en déclin depuis l'essor de Kubernetes
- Pas de support Helm
- Moins adapté aux architectures microservices complexes
- Peu de support des grands cloud providers (AWS, GCP, Azure)

K3S : Kubernetes léger

K3S est une distribution allégée de Kubernetes, créée par Rancher/SUSE. Elle regroupe toutes les fonctionnalités de Kubernetes dans un seul binaire de 70MB, optimisé pour les environnements contraints.

Installer K3S en une commande : `curl -sfL https://get.k3s.io | sh -`

Déployer une application : `kubectl apply -f deployment.yaml`

Installer une app complexe avec Helm : `helm install mon-wordpress bitnami/wordpress`

Avantages :

- Toutes les fonctionnalités de Kubernetes (RBAC, Helm, PV/PVC, Ingress...)
- Léger , tourne sur des machines avec seulement 512MB de RAM
- Installation en une seule commande
- Compatible avec tous les outils de l'écosystème Kubernetes
- Haute disponibilité native avec rescheduling automatique
- Idéal pour IoT, edge computing et environnements de formation
- Supporté activement par Rancher/SUSE
- Standard de l'industrie ,compétence très demandée

Inconvénients :

- Plus complexe que Docker Swarm à apprendre
- Courbe d'apprentissage plus importante
- Sur-dimensionné pour une application simple sur une seule machine
- Consomme plus de ressources que Docker seul
- Fichiers de configuration YAML verbeux

Dans quel cas utiliser chaque technologie ?

Utiliser Docker seul quand :

- Tu développes en local sur ta machine
- Tu as une petite application sur un seul serveur
- Tu débutes avec les conteneurs
- Tu n'as pas besoin de haute disponibilité

Exemple concret : Un développeur qui teste son application en local, un petit blog sur un VPS, un environnement de développement.

Utiliser Docker Swarm quand :

- Tu as déjà une infrastructure Docker et tu veux la clustériser rapidement
- Ton équipe maîtrise Docker Compose et ne veut pas tout réapprendre
- Tu as une application simple avec peu de microservices
- Tu as des ressources limitées pour la formation de l'équipe

Exemple concret : Une PME avec 2-3 serveurs, une application web simple avec une base de données, une équipe de 2-3 développeurs.

Utiliser K3S quand :

- Tu veux toutes les fonctionnalités de Kubernetes sans la complexité d'un cluster full K8s
- Tu travailles sur des environnements contraints (IoT, Raspberry Pi, edge)
- Tu veux un environnement de formation/lab Kubernetes
- Tu prépares une migration vers Kubernetes en production
- Tu as besoin de RBAC, Helm, et de fonctionnalités avancées

Exemple concret : Un lab de formation (comme ce projet !), un cluster sur Raspberry Pi, une startup qui prépare sa montée en charge, un environnement de pré-production Kubernetes.

Utiliser Kubernetes full (EKS, GKE, AKS) quand :

- Tu as une application critique en production à grande échelle
- Tu as une équipe DevOps dédiée
- Tu as besoin d'autoscaling avancé (HPA, VPA, Cluster Autoscaler)
- Tu es sur un cloud provider (AWS, GCP, Azure)

Exemple concret : Netflix, Airbnb, Spotify — applications avec des millions d'utilisateurs nécessitant une disponibilité 99.99%.

